

collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 2 roman: "II" title: "A2A — Comunicação entre Agentes" subtitle: "Agent-to-Agent: como agentes descobrem, negociam e cooperam em tempo real." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA

Capa

Volume II — A2A — Comunicação entre Agentes

Agent-to-Agent: como agentes descobrem, negociam e cooperam em tempo real.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Como dois agentes se reconhecem e dividem trabalho sem humano no meio?

Sumário

1. Por que MCP não basta: a fronteira entre tool e agente
2. Agent Card: o "passaporte" do agente
3. O ciclo de vida de uma Task A2A
4. Streaming, long-running e webhooks de retorno
5. Modalidades: texto, dados, áudio, vídeo, formulários
6. Push notifications e operações assíncronas longas
7. Multi-turno: como agentes mantêm conversa sem confundir contexto
8. Composição: agente cliente, agente servidor, agente híbrido

- 9. A2A vs MCP: quando escolher cada um
- 10. Manifesto: o agente não é uma tool com mais tokens

1. Por que MCP não basta: a fronteira entre tool e agente

MCP resolve um lado da equação: o agente fala com **ferramentas** padronizadas. Mas há um problema que MCP **propositalmente não resolve**: e quando o outro lado da conexão não é uma ferramenta — é outro agente, com seu próprio LLM, suas próprias decisões, seu próprio estado interno?

A diferença operacional é grande:

- Uma **tool** retorna em milissegundos, é stateless, devolve dado.
- Um **agente** pode levar minutos, mantém estado entre chamadas, pode fazer perguntas de volta, pode escalar para um humano, pode falhar de formas semânticas (não só sintáticas).

Tratar agente como tool é a causa #1 de sistemas multi-agente que parecem funcionar no demo e desmoronam em produção. O **A2A** (Agent-to-Agent), publicado pela Google em abril de 2025 e endossado por mais de 50 parceiros, nasceu exatamente para preencher essa lacuna. Ele responde a quatro perguntas que MCP deixa em aberto:

1. **Como descobrir** que existe outro agente capaz de fazer X?
2. **Como acordar** o contrato da tarefa antes de iniciar?
3. **Como acompanhar** uma execução que dura horas?
4. **Como conversar** durante a execução (perguntas, follow-ups, cancelamento)?

A2A é deliberadamente **complementar a MCP**, não competitivo. A regra mental: MCP para falar com o mundo; A2A para falar com pares.

2. Agent Card: o "passaporte" do agente

Todo agente A2A publica um documento JSON em uma URL bem conhecida: `https://<dominio>/.well-known/agent.json`. Esse é o **Agent Card** — o passaporte que descreve quem o agente é, o que sabe fazer, como autenticar e quais modalidades aceita.

3. O ciclo de vida de uma Task A2A

A unidade canônica de A2A não é "uma mensagem" — é uma **Task**. Tasks têm identidade, estado, histórico e podem viver minutos, horas ou dias.

Os estados canônicos da máquina de estados de uma Task:



- **submitted** — task aceita, fila ou inicialização.
- **working** — execução ativa, pode emitir updates.
- **input-required** — agente pausou e precisa de resposta para continuar (este é o que separa A2A de uma fila de jobs comum).
- **completed** — sucesso, artefatos disponíveis.
- **failed** — erro irrecuperável.
- **canceled** — cliente solicitou cancelamento.

Transições inválidas (ex.: `completed` → `working`) **devem ser rejeitadas pelo servidor**, não silenciadas. Um agente que aceita reabrir tarefas fechadas é um agente sem máquina de estados — e por consequência sem contrato auditável.

4. Streaming, long-running e webhooks de retorno

A2A suporta três modos de execução, e escolher o errado degrada o sistema inteiro:

1. **Síncrono (`tasks/send`)** — para tarefas <30 segundos. Cliente espera, recebe resultado final. Simples, mas amarra conexão.
2. **Streaming (`tasks/sendSubscribe` , **SSE**)** — para tarefas onde o agente quer mostrar progresso. O cliente abre SSE, recebe eventos de `TaskStatusUpdateEvent` e `TaskArtifactUpdateEvent` até `final: true`.

Digital Tool	Percentage of Respondents
Email	85%
Video Conferencing	72%
Project Management Software	68%
Cloud Storage	65%
Instant Messaging	95%

Device Type	Smartphone (%)	Tablet (%)
Smartphone	45	30
Tablet	30	25
Smart TV	25	20

3. Push assíncrono (`pushNotificationConfig`)

— para tarefas que duram horas ou dias. Cliente registra um webhook; agente notifica via POST quando houver transição. Indispensável para integrações onde o cliente não pode manter conexão aberta (mobile, serverless).

Anti-padrão crônico: usar streaming SSE para tarefas de 6 horas. SSE não foi desenhado para isso; a primeira interrupção de rede e a task vira fantasma. Para >2 minutos, use push.

5. Modalidades: texto, dados, áudio, vídeo, formulários

Mensagens A2A são compostas por **Parts**, e cada Part declara seu tipo. Esse é o ponto onde A2A se distancia mais radicalmente de protocolos baseados em "chat":

[illegible]



Tipos canônicos de Part:

- **text** — string simples.
- **file** — binário, com bytes inline ou URI.
- **data** — JSON estruturado (parâmetros, metadados, formulários).

A inovação prática é o `DataPart`: o agente cliente pode mandar **estruturas semânticas** sem serializar em texto e torcer para o agente servidor parsear. E o agente servidor pode **retornar formulários estruturados** que o cliente renderiza nativamente, em vez de devolver markdown que o front-end precisa interpretar.

6. Push notifications e operações assíncronas longas

Para tarefas de longa duração, o cliente registra um endpoint de callback:



O servidor então, a cada transição relevante, faz POST autenticado para essa URL. Três regras de produção que não estão no spec mas separam quem leva A2A a sério:

1. **HMAC assinado por payload**, não só bearer no header. Bearer pode vazar; HMAC com timestamp tem janela curta de replay.
2. **Idempotência via `eventId`**. O cliente deve deduplicar; servidores reentregam em caso de falha de rede.
3. **Backoff exponencial com circuit breaker**. Se o webhook do cliente está caindo 5xx por 10 minutos, pare de tentar e marque a notificação como `delivery-failed`, mas mantenha a task disponível para polling.

7. Multi-turno: como agentes mantêm conversa sem confundir contexto

O conceito que muitos primeiro-implementadores quebram é o **sessionId vs taskId**. Eles **não são** sinônimos:

- **sessionId** — conversa lógica entre dois agentes. Pode conter N tasks.
- **taskId** — uma unidade de trabalho. Tem ciclo de vida finito.

Exemplo: um agente de viagens recebe `sessionId=s-1`. Dentro dessa sessão, ele cria `task=t-1` para "achar voos", `task=t-2` para "reservar hotel", `task=t-3` para "gerar roteiro". Todas as três tasks compartilham contexto de sessão (destino, datas, perfil do viajante), mas têm artefatos e estado independentes.

Quando uma task entra em `input-required`, o cliente deve responder com mensagem que **referencia o taskId**, não cria outra task. Criar nova task para responder uma pergunta é o erro mais comum em integrações iniciais — e quebra o histórico em pedaços não-relacionáveis.

8. Composição: agente cliente, agente servidor, agente híbrido

A2A é simétrico: todo agente pode ser **servidor** (expõe Agent Card e recebe tasks) e **cliente** (consome tasks de outros agentes). Três padrões de composição que se repetem em produção:

Padrão Orchestrator

Um agente "maestro" recebe a task do humano e delega sub-tasks para N agentes especialistas. O orchestrator agrega artefatos e devolve resultado único. Funciona bem quando a decomposição é estável.

Padrão Pipeline

Agente A → agente B → agente C, cada um adicionando valor sobre o artefato do anterior. Cada link é uma task A2A separada. Robusto porque cada nó pode ser reescalado, substituído ou observado isoladamente.

Padrão Marketplace

Um broker recebe uma necessidade, faz **discovery** entre Agent Cards indexados, escolhe o melhor agente para cada skill (por reputação, custo, latência) e delega. É o padrão mais ambicioso e o que mais exige infraestrutura adicional (registry, trust layer — ver Volume VII).

9. A2A vs MCP: quando escolher cada um

Uma das confusões mais comuns: "se já uso MCP, preciso de A2A?". A resposta honesta exige uma matriz de decisão:

Critério	Use MCP	Use A2A
O outro lado tem seu próprio LLM ?	Não	Sim
Resposta esperada em <1s ?	Sim	Não
Há state interno persistente no outro lado?	Não	Sim
Pode haver input-required mid-execution?	Não	Sim
O outro lado pode escalar para humano ?	Não	Sim
Quero descoberta dinâmica ?	Não	Sim
Quero descrever uma função estática ?	Sim	Não

Caso real: um agente de atendimento usa **MCP** para falar com o CRM (Resource: ficha do cliente; Tool: registrar nota) e **A2A** para chamar o agente de cobrança da outra área quando precisa renegociar uma fatura. A regra mental: MCP é integração; A2A é colaboração.

10. Manifesto: o agente não é uma tool com mais tokens

A indústria passou 2024 inteiro tentando empurrar agentes para a abstração de tool-call. Foi reconfortante porque cabia no mental model existente (function calling) — e foi ineficaz, porque tools não têm estado, não fazem perguntas de volta e não negociam.

A2A inverte a metáfora: o agente é um **par**, não um subordinado. Ele pode recusar, pode pedir clarificação, pode entregar parcial, pode escalar. Quem fala com agente como se fosse stored procedure colhe sistemas frágeis.

Tese final do volume: A arquitetura multi-agente real só começa quando paramos de tratar o outro agente como uma tool com mais tokens — e começamos a tratá-lo como uma entidade com contrato, máquina de estados e histórico próprio. A2A é o primeiro protocolo que opera essa virada.

Checklist Canônico — A2A em produção

- [] Agent Card publicado em `/.well-known/agent.json` e versionado.
- [] Skills com descrição clara, exemplos e tags estáveis.
- [] Máquina de estados de Task rejeita transições inválidas.
- [] Streaming usado apenas para tarefas <2 min; push para o resto.
- [] Push notifications assinadas com HMAC + janela de replay curta.
- [] sessionId separado de taskId em toda a base de código.
- [] Resposta a `input-required` referencia taskId, não cria task nova.
- [] DataPart usado para dados estruturados (não serializar em texto).
- [] Cancelamento (`tasks/cancel`) testado e idempotente.
- [] Discovery (catálogo de Agent Cards) com TTL e revalidação.

Glossário do Volume

- **Agent Card** — JSON em `/.well-known/agent.json` que descreve o agente.
- **Skill** — unidade de competência exposta no Agent Card.
- **Task** — unidade de trabalho com máquina de estados própria.
- **Session** — conversa lógica que agrupa múltiplas tasks.
- **Part** — pedaço tipado de uma mensagem (text, file, data).
- **input-required** — estado em que a task pausa aguardando resposta do cliente.
- **Push Notification** — webhook que o servidor chama para informar transições.
- **Streaming (A2A)** — entrega de updates via SSE durante execução.

Gancho para o Próximo Volume

A2A resolve **um agente conversando com outro**. Mas a indústria não para por aí: a IBM publicou ACP, a OpenAI tem propostas próprias, a Microsoft empurra AutoGen como padrão de fato. Como escolher entre eles? Como construir camadas de abstração que sobrevivam à guerra de protocolos? Esse é o terreno do **Volume III — ACP e Padrões de Interoperabilidade**.

NEXUS PROTOCOL · Volume II · Edição Canônica 1.0.0 · 2026-06-08 MMN AI-to-AI ·
Nexus HUB57 · Ecossistema MMN AI-to-AI